

Timothy Song

Professor Shim

DATA 260

31 August 2026

Homework 1

GitHub Repository: [data260-9981](#)

Section 0 – Personal Configuration and Domain

Hardware: MacBook Pro, Apple M4 Pro, 14 cores, 24 GB RAM

Local Model: qwen3:8b

Tagged Commit Hash: Please check submitted version on Canvas

SID4: 9981

PORT_BASE: 8081

PREFIX: s9981

SEED: 9981

VERIFY_SEED: 269981

DOMAIN_ID: 5 (Local restaurant inspections)

Questions (Part 1):

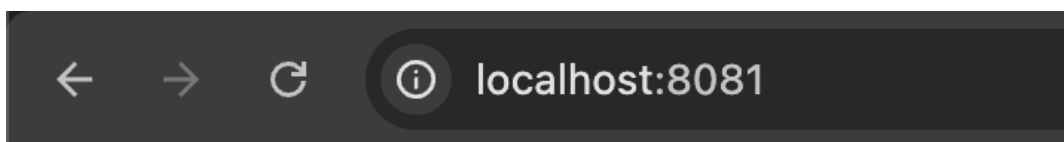
I. HTML

3. Before implementing the HTML code, I created the DOMAIN_SCHEMA.md file that defined my domain entity, along with its fields and category values.

```
DOMAIN_SCHEMA.md > abc # Domain Entity > abc ## Fields and category values
1  # Domain Entity
2
3  Local Restaurant Inspection
4
5  ## Fields and category values
6
7  Primary field: Restaurant name
8  Secondary field: Restaurant address
9  Email input: Submitter's email
10 Content/description field: Inspection Summary
11 Category selection: Routine Inspection, Follow-Up Inspection, Complaint Inspection, Reinstatement Inspection
12 Checkbox: I agree to the terms and conditions.
13 Submit button: Submit inspection
```

1. I created an HTML page titled HW1-Timothy Song
2. I added a heading tag that named my assigned domain entity, “Local Restaurant Inspections”, using the heading tag “<h1>”, which renders the biggest font size.
4. I created a form for my domain entity following the specified criteria in a-g.
 - a) I added “restaurantName”, the required text input for the entity's primary field, with a placeholder describing it. The cursor automatically focuses on this field when the page loads.
 - b) I added “restaurantAddress”, the required text input for the entity's secondary field, with a descriptive placeholder.
 - c) I added “submitterEmail”, a required email input for the submitter's email, with a descriptive placeholder.
 - d) I added “inspectionSummary”, a required text area for the entity's content/description field, with a descriptive placeholder.
 - e) I added “category”, a dropdown for category selection, with four domain-appropriate options.
 - f) I added “terms”, a checkbox and a label with the text "I agree to the terms and conditions."
 - g) I added “submit”, a submit button with domain-appropriate text.
5. I added a script tag at the end of my HTML file to link my JavaScript code for Part II.

```
code > web_application > <> index.html > <html>
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4 <<title>HW1-Timothy Song</title>
5 </head>
6 <body>
7 <h1>Local Restaurant Inspections</h1>
8 <form id="inspectionForm">
9 <label for="restaurantName">What is the restaurant's name?</label>
10 <input type="text" id="restaurantName" name="restaurantName" placeholder="Please enter the restaurant's name..." required autofocus><br><br>
11 <label for="restaurantAddress">What is the restaurant's address?</label>
12 <input type="text" id="restaurantAddress" name="restaurantAddress" placeholder="Please enter the restaurant's address..." required><br><br>
13 <label for="submitterEmail">What is the submitter's email?</label>
14 <input type="email" id="submitterEmail" name="submitterEmail" placeholder="Please enter the submitter's email..." required><br><br>
15 <label for="inspectionSummary">Please give a summary of the inspection:</label>
16 <textarea id="inspectionSummary" name="inspectionSummary" placeholder="Please enter the inspection summary..." required></textarea><br><br>
17 <label for="category">Please select the type of inspection:</label>
18 <select id="category" name="category">
19 <option value="routine">Routine Inspection</option>
20 <option value="follow-up">Follow-Up Inspection</option>
21 <option value="complaint">Complaint Inspection</option>
22 <option value="reinstatement">Reinstatement Inspection</option>
23 </select><br><br>
24 <input type="checkbox" id="terms" name="terms">
25 <label for="terms">I agree to the terms and conditions.</label><br><br>
26 <input type="submit" value="Submit inspection">
27 </form>
28 <script src="script.js"></script>
29 </body>
30 </html>
```



<

Local Restaurant Inspections

What is the restaurant's name?

What is the restaurant's address?

What is the submitter's email?

Please give a summary of the inspection:

Please select the type of inspection:

☐ I agree to the terms and conditions.

II. JavaScript

1. I wrote a JavaScript function, “validateField”, using an arrow function to validate:
 - a) The content field has more than 25 characters and if validation fails, an alert is displayed with an appropriate message.
 - b) The terms and conditions checkbox is checked and if validation fails, an alert is displayed with an appropriate message.
2. I converted the form data into a JSON string and logged the output in the console after successful form submission.
3. I used object destructuring to extract the primary field and email field from the parsed object and log their values in the console.
4. I used the spread operator to add a new field, “submissionDate”, with the current date and time to the parsed object and log the updated parsed object in the console.
5. I created a closure to track how many times the form has been successfully submitted and log the submission count each time the form is submitted.

```
code > web_application > JS script.js > ...
1  const validateField = () => {
2    const content = document.getElementById("inspectionSummary").value;
3    if (content.length <= 25) {
4      alert("The inspection summary has to have more than 25 characters.");
5      return false;
6    }
7    const checked = document.getElementById("terms").checked;
8    if (!checked) {
9      alert("You have to agree to the terms and conditions.");
10     return false;
11   }
12   return true;
13 };
14 const form = document.getElementById("inspectionForm");
15 const createCounter = () => {
16   let count = 0;
17   return () => {
18     count++;
19     return count;
20   }
21 };
22 const counter = createCounter();
23 form.addEventListener("submit", (event) => {
24   event.preventDefault();
25   if (validateField()) {
26     const formData = {
27       restaurantName: document.getElementById("restaurantName").value,
28       restaurantAddress: document.getElementById("restaurantAddress").value,
29       submitterEmail: document.getElementById("submitterEmail").value,
30       inspectionSummary: document.getElementById("inspectionSummary").value,
31       category: document.getElementById("category").value,
32       terms: document.getElementById("terms").checked
33     };
34     const jsonString = JSON.stringify(formData);
35     console.log(jsonString);
36     const parsedObject = JSON.parse(jsonString);
37     const {restaurantName, submitterEmail} = parsedObject;
38     console.log(restaurantName);
39     console.log(submitterEmail);
40     const updatedParsedObject = {
41       ...parsedObject,
42       submissionDate: new Date().toISOString()
43     };
44     console.log(updatedParsedObject);
45     const count = counter();
46     console.log("The submission count is: ", count);
47   }
48 });
```

*Validation check for 1a:

<

Local Restaurant Inspections

What is the restaurant's name?

O2 Valley

What is the restaurant's address?

452 University Avenue

What is the submitter's email?

timothy.song@sjsu.edu

Please give a summary of the inspection:

Passed inspection

Please select the type of inspection:

Routine Inspection

☒ I agree to the terms and conditions.

Submit inspection

localhost:8081 says

The inspection summary has to have more than 25 characters.

OK

*Validation check for 1b:

<

Local Restaurant Inspections

What is the restaurant's name?

O2 Valley

What is the restaurant's address?

452 University Avenue

What is the submitter's email?

timothy.song@sjsu.edu

Please give a summary of the inspection:

The restaurant passed the inspection with flying colors.

Please select the type of inspection:

Routine Inspection

☐ I agree to the terms and conditions.

Submit inspection

localhost:8081 says

You have to agree to the terms and conditions.

OK

*Console logging output and counter check

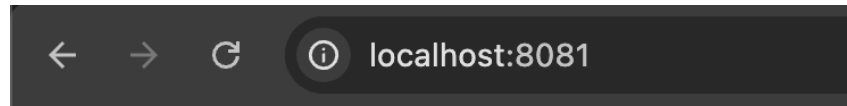
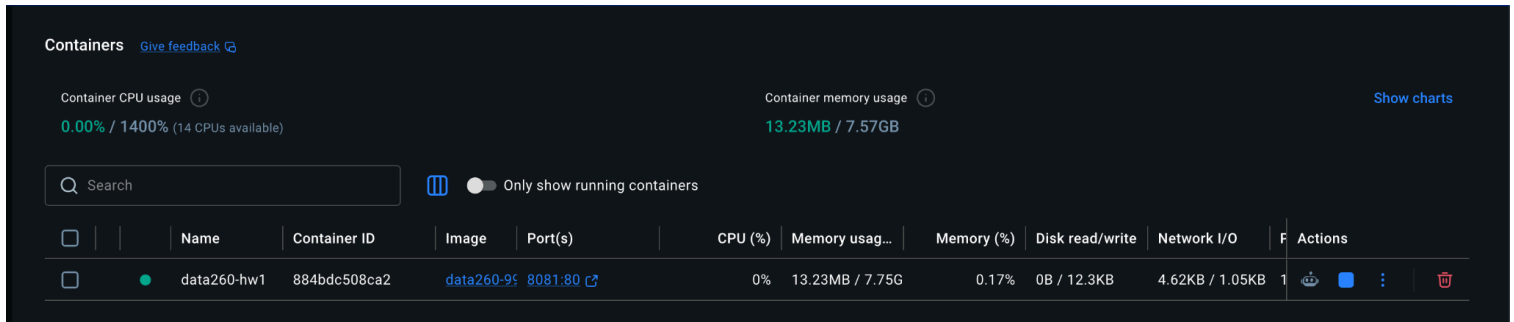
```

{"restaurantName":"O2 Valley","restaurantAddress":"452 University Avenue","submitterEmail":"timothy.song@sjsu.edu","inspectionSummary":"The restaurant passed the inspection with flying colors.","category":"routine","terms":true}
O2 Valley
timothy.song@sjsu.edu
> {restaurantName: 'O2 Valley', restaurantAddress: '452 University Avenue', submitterEmail: 'timothy.song@sjsu.edu', inspectionSummary: 'The restaurant passed the inspection with flying colors.', category: 'routine', ...}
The submission count is: 1
{"restaurantName":"Pho Nam","restaurantAddress":"844 W El Camino Real","submitterEmail":"timothy.song@sjsu.edu","inspectionSummary":"This restaurant passed the follow-up inspection","category":"follow-up","terms":true}
Pho Nam
timothy.song@sjsu.edu
> {restaurantName: 'Pho Nam', restaurantAddress: '844 W El Camino Real', submitterEmail: 'timothy.song@sjsu.edu', inspectionSummary: 'This restaurant passed the follow-up inspection', category: 'follow-up', ...}
The submission count is: 2

```

Deployment:

I created a Docker image of the application and built and ran the application using docker locally.



<

Local Restaurant Inspections

What is the restaurant's name?

What is the restaurant's address?

What is the submitter's email?

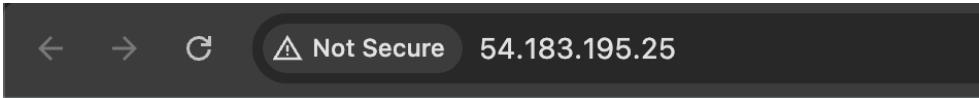
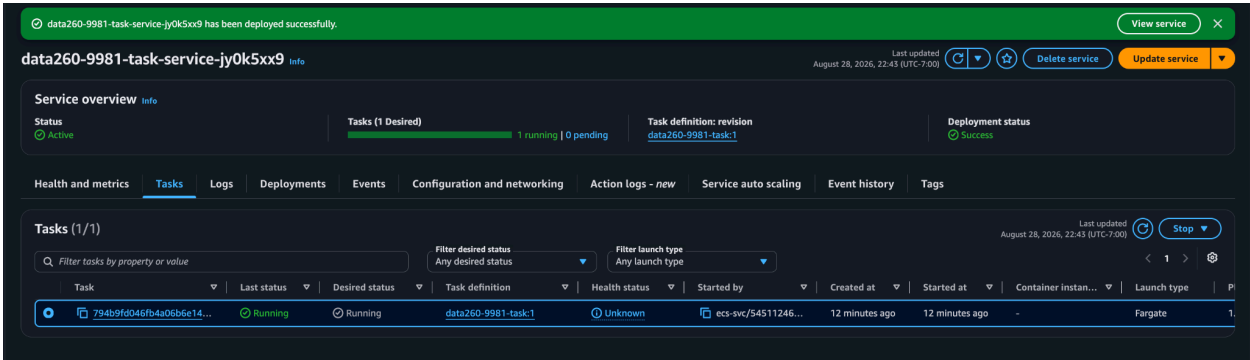
Please give a summary of the inspection:

Please select the type of inspection:

☐ I agree to the terms and conditions.

I created an AWS ECS service with one task running the above application using a docker image which is accessible on my public IP address.

```
code > Dockerfile
1 FROM nginx:alpine
2 COPY web_application/ /usr/share/nginx/html/
3 EXPOSE 80
```



<

Local Restaurant Inspections

What is the restaurant's name?

What is the restaurant's address?

What is the submitter's email?

Please give a summary of the inspection:

Please select the type of inspection:

☐ I agree to the terms and conditions.

Agentic AI (Part 2):

Requirements:

1. I am using Python version 3.12.14.

```
[(data260) optim@Timothys-MacBook-Pro ~ % python --version
Python 3.12.14
```

2. Ollama is installed and running with model qwen3:8b.

```
[(data260) optim@Timothys-MacBook-Pro ~ % ollama --version
ollama version is 0.33.2
[(data260) optim@Timothys-MacBook-Pro ~ % ollama list
NAME          ID          SIZE      MODIFIED
qwen3:8b      500a1f067a9f  5.2 GB    About a minute ago
[(data260) optim@Timothys-MacBook-Pro ~ %
```

3. I created agents_demo.py that contains a planner and a reviewer agent as well as a finalization step that all use JSON format for the output.

```
agents_demo.py > ...
1  import json
2  from langchain_ollama import ChatOllama
3  model = ChatOllama(
4      model="qwen3:8b",
5      temperature=0.0,
6      format="json"
7  )
8  title = input("Please enter a title... ")
9  content = input("Please enter the content... ")
10  planner_prompt = f"""
11  You are the Planner agent, so please analyze the title and content.
12  Title: {title}
13  Content: {content}
14  Please give me three tags and a one sentence summary up to twenty five words.
15  """
16  response = model.invoke(planner_prompt)
17  planner_output = json.loads(response.content)
18  print("Planner Output")
19  print(json.dumps(planner_output, indent=2))
20  reviewer_prompt = f"""
21  You are the Reviewer agent, so please review the Planner output.
22  Title: {title}
23  Content: {content}
24  Planner output: {json.dumps(planner_output)}
25  Verify that there are 3 tags that are relevant to the title and content.
26  Also please make sure that the summary is correct and does not exceed twenty five words.
27  If there is anything wrong, please fix it.
28  """
29  reviewer_response = model.invoke(reviewer_prompt)
30  reviewer_output = json.loads(reviewer_response.content)
31  print("Reviewer Output")
32  print(json.dumps(reviewer_output, indent=2))
33  final_output = {
34      "tags": reviewer_output["tags"][:3],
35      "summary": " ".join(reviewer_output["summary"].split()[:25])
36  }
37  print("Finalized Output")
38  print(json.dumps(final_output, indent=2))
```

```

• (data260) optim@Timothys-MacBook-Pro data260-9981 % python agents_demo.py
Please enter a title... O2 Valley
Please enter the content... O2 Valley passed the inspection with flying colors. Their food handling, food safety, hygiene, quality, and health standards all exceeded our standards.
Planner Output
{
  "tags": [
    "Inspection Passed",
    "Food Safety Excellence",
    "Hygiene Standards Met"
  ],
  "summary": "O2 Valley passed inspection with flying colors, exceeding food safety, hygiene, quality, and health standards."
}
Reviewer Output
{
  "tags": [
    "Inspection Passed",
    "Food Safety Excellence",
    "Hygiene Standards Met"
  ],
  "summary": "O2 Valley passed inspection with flying colors, exceeding food safety, hygiene, quality, and health standards."
}
Finalized Output
{
  "tags": [
    "Inspection Passed",
    "Food Safety Excellence",
    "Hygiene Standards Met"
  ],
  "summary": "O2 Valley passed inspection with flying colors, exceeding food safety, hygiene, quality, and health standards."
}
○ (data260) optim@Timothys-MacBook-Pro data260-9981 %

```

4. I did not hardcode my domain into the agent logic. The tags and summary are derived from the title and content.

Deliverables

1. python agents_demo.py as shown in the screenshot above.
2. Shown in the screenshot above.
3.
 1. The 3 final tags I obtained were “Inspection Passed”, “Food Safety Excellence”, and “Hygiene Standards Met”.
 2. The final summary was "O2 Valley passed inspection with flying colors, exceeding food safety, hygiene, quality, and health standards."
 3. No, the reviewer did not change anything because the planner agent had already done its job of generating three relevant tags and a summary of appropriate length.
4.
 - Planner Step: The planner agent takes my title and content to generate three relevant tags and a summary that does not exceed twenty five words.
 - Reviewer Step: The reviewer agent reviews the planner agent’s output to verify that there are three relevant tags as well as a correct summary that does not exceed twenty five words. If anything is wrong, the reviewer agent will fix it.
 - Finalizer Step: The finalizer limits the reviewer output to three tags and a twenty five word summary, then returns the final output in JSON format.

Measuring Non Determinism (Part 3)

I chose one fixed domain-relevant title and content. I saved it once and left it unchanged, as reports/hw01/cases/nondeterminism_input.json.

```
reports > hw01 > cases > {} nondeterminism_input.json > ...
1  {
2    "title": "02 Valley Routine Inspection",
3    "content": "02 Valley passed the routine inspection. Their dining area, kitchen, freezer, and prep station were all up
4    to standards."
}
```

1. I modified agents_demo.py to put the planner, reviewer, and finalizer into a pipeline function, which allowed me to call it repeatedly to make the 40 total runs easier. I created run_pipeline.py to automate the run repetitions and record and log the specified criteria.

```
agents_demo.py > ...
1  import json
2  from langchain_ollama import ChatOllama
3  def pipeline(title, content, temperature):
4      model=ChatOllama(
5          model="qwen3:8b",
6          temperature=temperature,
7          format="json"
8      )
9      planner_prompt = f"""
10     You are the Planner agent, so please analyze the title and content.
11     Title: {title}
12     Content: {content}
13     Please give me three tags and a one sentence summary up to twenty five words.
14     """
15     response = model.invoke(planner_prompt)
16     planner_output = json.loads(response.content)
17     print("Planner Output")
18     print(json.dumps(planner_output, indent=2))
19     reviewer_prompt = f"""
20     You are the Reviewer agent, so please review the Planner output.
21     Title: {title}
22     Content: {content}
23     Planner output: {json.dumps(planner_output)}
24     Verify that there are 3 tags that are relevant to the title and content.
25     Also please make sure that the summary is correct and does not exceed twenty five words.
26     If there is anything wrong, please fix it.
27     """
28     reviewer_response = model.invoke(reviewer_prompt)
29     reviewer_output = json.loads(reviewer_response.content)
30     print("Reviewer Output")
31     print(json.dumps(reviewer_output, indent=2))
32     final_output = {
33         "tags": reviewer_output["tags"][:3],
34         "summary": " ".join(reviewer_output["summary"].split()[:25])
35     }
36     print("Finalized Output")
37     print(json.dumps(final_output, indent=2))
38     return final_output
39  if __name__ == "__main__":
40     title = input("Please enter a title... ")
41     content = input("Please enter the content... ")
42     pipeline(title, content, 0.0)
```

```

run_pipeline.py > ...
1  import json
2  import time
3  from agents_demo import pipeline
4  from collections import Counter
5  import numpy as np
6  with open(
7      "reports/hw01/cases/nondeterminism_input.json",
8      "r"
9  ) as file:
10     fixed_input = json.load(file)
11     title = fixed_input["title"]
12     content = fixed_input["content"]
13     results = []
14     for temperature in [0.7, 0.0]:
15         print(f"\nTemperature: {temperature}")
16         for trial in range(1, 21):
17             start_time = time.perf_counter()
18             result = pipeline(
19                 title,
20                 content,
21                 temperature
22             )
23             latency_ms = (
24                 time.perf_counter() - start_time
25             ) * 1000
26             print(
27                 f"Run {trial}: "
28                 f"{result['tags']} "
29                 f"{latency_ms:.2f} ms"
30             )
31             results.append({
32                 "temperature": temperature,
33                 "trial": trial,
34                 "tags": result["tags"],
35                 "latency_ms": latency_ms
36             })
37     with open(
38         "reports/hw01/raw/nondeterminism_results.json",
39         "w"
40     ) as file:
41         json.dump(results, file, indent=2)
42     for temperature in [0.7, 0.0]:
43         temp_results = [
44             r for r in results
45             if r["temperature"] == temperature
46         ]
47         tag_sets = {
48             tuple(sorted(r["tags"]))
49             for r in temp_results
50         }
51         tag_counts = Counter()
52         for r in temp_results:
53             for tag in set(r["tags"]):
54                 tag_counts[tag] += 1
55         tags_all_20 = [
56             tag
57             for tag, count in tag_counts.items()
58             if count == 20
59         ]
60         tags_once = [
61             tag
62             for tag, count in tag_counts.items()
63             if count == 1
64         ]
65         latencies = [
66             r["latency_ms"]
67             for r in temp_results
68         ]
69         p50 = np.percentile(latencies, 50)
70         p95 = np.percentile(latencies, 95)
71         p99 = np.percentile(latencies, 99)
72         print(f"\nThe results for temperature {temperature}")
73         print(f"The distinct tag sets: {len(tag_sets)}")
74         print(f"The tags in all 20 runs: {tags_all_20}")
75         print(f"The tags in exactly 1 run: {tags_once}")
76         print(f"Latency for p50: {p50:.2f} ms")
77         print(f"Latency for p95: {p95:.2f} ms")
78         print(f"Latency for p99: {p99:.2f} ms")

```

2.

Metric	Temp 0.7	Temp 0.0
# of Distinct Tag Sets	18	1
Tags in all 20 runs	None	'Routine Inspection', 'O2 Valley', 'Standards Compliance'
Tags in exactly 1 run	'Kitchen Standards Met', 'Dining Area Compliance', '#RoutineInspection', '#InspectionPassed', '#O2Valley', 'Facility Standards Met', 'Hygiene Standards Met', 'Facility Compliance Verified', 'Facility Audit', 'Standards Compliance', 'Compliance Standards Met', 'Hygiene Standards', 'Operational Standards Met', 'Compliance', 'Facility Standards Verified', 'Sanitation Standards	None
Latency p50 / p95 / p99 (ms)	24278.31 42241.62 44770.98	26233.95 26550.50 28461.01

3. The results are saved as JSON in reports/hw01/raw/nondeterminism_results.json

```

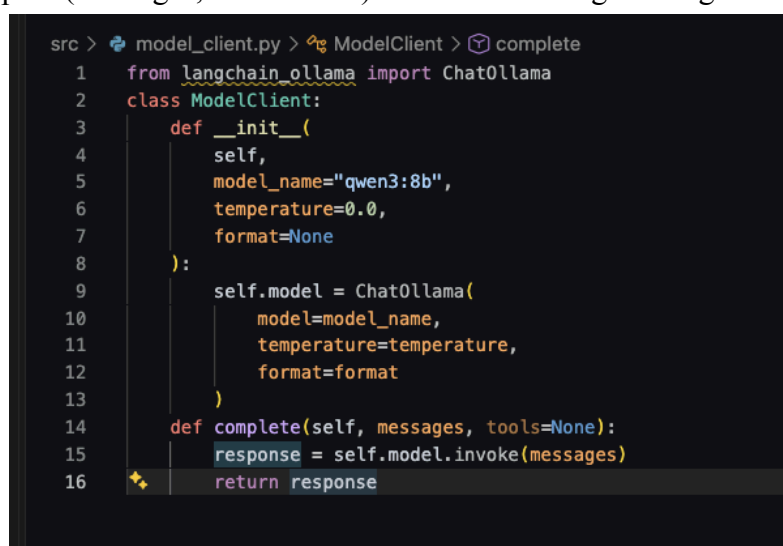
reports > hw01 > raw > {} nondeterminism_results.json > {} 5
1  {
2    {
3      "temperature": 0.7,
4      "trial": 1,
5      "tags": [
6        "Routine Inspection Passed",
7        "Dining Area Compliance",
8        "Kitchen Standards Met"
9      ],
10     "latency_ms": 42075.210999988485
11   },
12   {
13     "temperature": 0.7,
14     "trial": 2,
15     "tags": [
16       "#RoutineInspection",
17       "#O2Valley",
18       "#InspectionPassed"
19     ],
20     "latency_ms": 26444.304541975725
21   },
22   {
23     "temperature": 0.7,
24     "trial": 3,
25     "tags": [
26       "Routine Inspection Passed",
27       "O2 Valley Compliance",
28       "Facility Standards Met"
29     ],
30     "latency_ms": 19661.218624998583
31   },

```

4. Even if two users send identical inputs, they could receive different results if the temperatures are different. For example, in my runs using a temperature of 0.7, there were 18 distinct tag sets in 20 runs. So, two users can end up with different tags even with identical inputs. But, when I used a temperature of 0.0, I ended up with only 1 distinct tag set. This means that my results were consistent across all 20 runs using a temperature of 0.0. Run-to-run variation is acceptable when creating the tags, because different tags can still be accurate and relevant to the content. But, variation is not acceptable if it affects accuracy. For example, if the tags are like #InspectionPassed, #Standards Compliance, #Compliance, that variation is ok. However, tags such as #InspectionFailed or #Not Compliant are unacceptable.

Model Client and Token Accounting (Part 4)

1. I created a reusable model-adapter module at src/model_client.py, defining a stable interface complete(messages, tools=None). All model calls go through this adapter.



```
src > model_client.py > ModelClient > complete
1 from langchain_ollama import ChatOllama
2 class ModelClient:
3     def __init__(
4         self,
5         model_name="qwen3:8b",
6         temperature=0.0,
7         format=None
8     ):
9         self.model = ChatOllama(
10             model=model_name,
11             temperature=temperature,
12             format=format
13         )
14     def complete(self, messages, tools=None):
15         response = self.model.invoke(messages)
16         return response
```

2,3, 4,5. I created hw1_client.py as a small command-line demo that imports the adapter. After every model response, it prints the input tokens, output tokens, and total tokens for that turn. On exit, it prints the cumulative input tokens, output tokens, and turn count. I also added a /stats command that shows turn count, cumulative token counts, and serialized conversation-history length, without altering the history. I also created AGENT.md and asked for strict bullet-only code review.

```

hw1_client.py > ...
1 import json
2 from src.model_client import ModelClient
3 client = ModelClient()
4 with open("AGENT.md", "r") as file:
5     agent_instructions = file.read()
6 history = [
7     {
8         "role": "system",
9         "content": agent_instructions
10    }
11 ]
12 turn_count = 0
13 cumulative_input_tokens = 0
14 cumulative_output_tokens = 0
15 while True:
16     user_input = input("You: ")
17     if user_input.lower() in ["exit", "quit"]:
18         print("\nSession totals")
19         print(f"Turns: {turn_count}")
20         print(f"Input tokens: {cumulative_input_tokens}")
21         print(f"Output tokens: {cumulative_output_tokens}")
22         break
23     if user_input == "/stats":
24         history_length = len(
25             json.dumps(history)
26         )
27         print("\nStats")
28         print(f"Turns: {turn_count}")
29         print(f"Input tokens: {cumulative_input_tokens}")
30         print(f"Output tokens: {cumulative_output_tokens}")
31         print(f"History length: {history_length}")
32         continue
33     history.append({
34         "role": "user",
35         "content": user_input
36     })
37     response = client.complete(history)
38     input_tokens = response.usage_metadata["input_tokens"]
39     output_tokens = response.usage_metadata["output_tokens"]
40     total_tokens = response.usage_metadata["total_tokens"]
41     turn_count += 1
42     cumulative_input_tokens += input_tokens
43     cumulative_output_tokens += output_tokens
44     print(f"\nAssistant: {response.content}")
45     print(f"Input tokens: {input_tokens}")
46     print(f"Output tokens: {output_tokens}")
47     print(f"Total tokens: {total_tokens}")
48     history.append({
49         "role": "assistant",
50         "content": response.content
51     })

```

```

agents_demo.py > ...
1 import json
2 from src.model_client import ModelClient
3 def pipeline(title, content, temperature):
4     client = ModelClient(
5         model_name="qwen3:8b",
6         temperature=temperature,
7         format="json"
8     )
9     planner_prompt = f"""
10     You are the Planner agent, so please analyze the title and content.
11     Title: {title}
12     Content: {content}
13     Please give me three tags and a one sentence summary up to twenty five words.
14     """
15     response = client.complete(planner_prompt)
16     planner_output = json.loads(response.content)
17     print("Planner Output")
18     print(json.dumps(planner_output, indent=2))
19     reviewer_prompt = f"""
20     You are the Reviewer agent, so please review the Planner output.
21     Title: {title}
22     Content: {content}
23     Planner output: {json.dumps(planner_output)}
24     Verify that there are 3 tags that are relevant to the title and content.
25     Also please make sure that the summary is correct and does not exceed twenty five words.
26     If there is anything wrong, please fix it.
27     """
28     reviewer_response = client.complete(reviewer_prompt)
29     reviewer_output = json.loads(reviewer_response.content)
30     print("Reviewer Output")
31     print(json.dumps(reviewer_output, indent=2))
32     final_output = {
33         "tags": reviewer_output["tags"][:3],
34         "summary": " ".join(reviewer_output["summary"].split()[:25])
35     }
36     print("Finalized Output")
37     print(json.dumps(final_output, indent=2))
38     return final_output
39 if __name__ == "__main__":
40     title = input("Please enter a title... ")
41     content = input("Please enter the content... ")
42     pipeline(title, content, 0.0)

```

AGENT.md

1 Please use strict bullet-only code review.

* Verified the model follows strict bullet-only code review.

```

(base) optim@Timothys-MacBook-Pro data260-9981 % python hw1_client.py
You: Please take a look at this code: for x in range(5): print("Hello!")

Assistant: - The loop variable `x` is declared but not used; it can be removed for clarity.
- The `print("Hello!")` statement lacks parentheses, which is invalid in Python 3.
- The loop runs 5 times, but the output is redundant and could be simplified.
- No error handling or input is included, making the code static and limited in scope.
- The code is syntactically correct in Python 2 but will fail in Python 3 due to the `print` syntax.
Input tokens: 42
Output tokens: 747
Total tokens: 789
You: 

```

6. I ran a five-turn conversation and recorded the /stats output after turn 3 and after turn 5 and after exiting.

```

Stats
Turns: 3
Input tokens: 503
Output tokens: 1924
History length: 1823

```

```

Stats
Turns: 5
Input tokens: 1540
Output tokens: 3318
History length: 3533

```

```

History length: 3533
You: exit

Session totals
Turns: 5
Input tokens: 1540
Output tokens: 3318

```

7.

- Why is prior conversation context resent with every turn?
 - Prior conversation context is resent with every turn, so the model can refer to previous history to help it with generating its responses.
- How is a system prompt different from a user message?
 - System prompts tell the model how to do something, while user messages tell the model what to do.
- Why do input tokens grow over a conversation?
 - Input tokens grow over a conversation because with each new message the history is resent too, so the token amount grows.
- What eventually limits that growth?
 - Usually, the model's context window will limit the growth, because each model has a maximum number of tokens.